

Introduction to Multilayer Perceptron

Contents

- ✓ Introduction to Multilayer perceptron
- ✓ Comparison between Single layer and multilayer perceptron
- ✓ Feed Forward Neural Network
- ✓ Shallow vs Deep Neural Networks

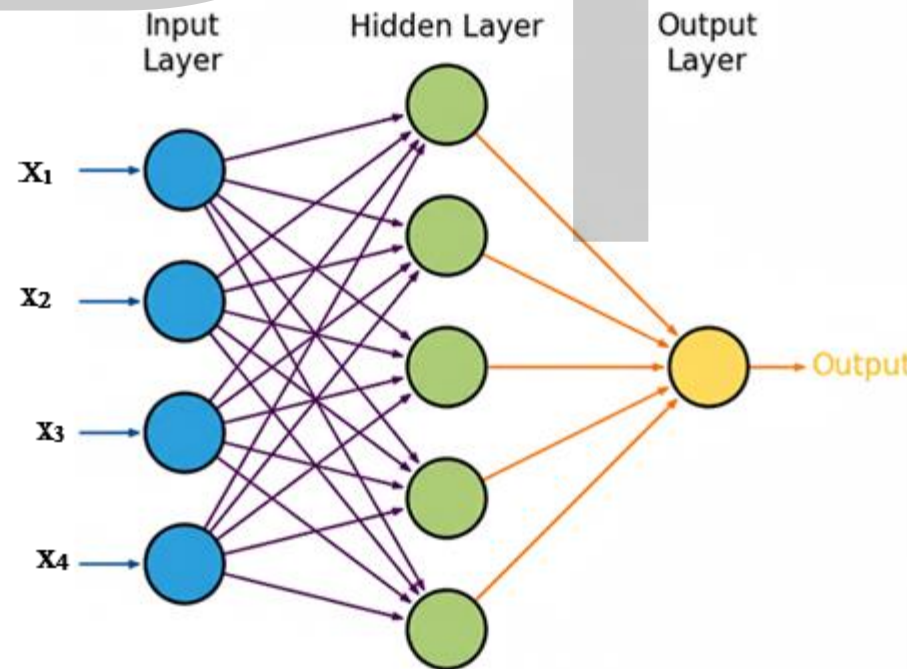
Introduction to Multilayer Perceptron

Multilayer Perceptron (MLP) is a supervised feed forward neural network

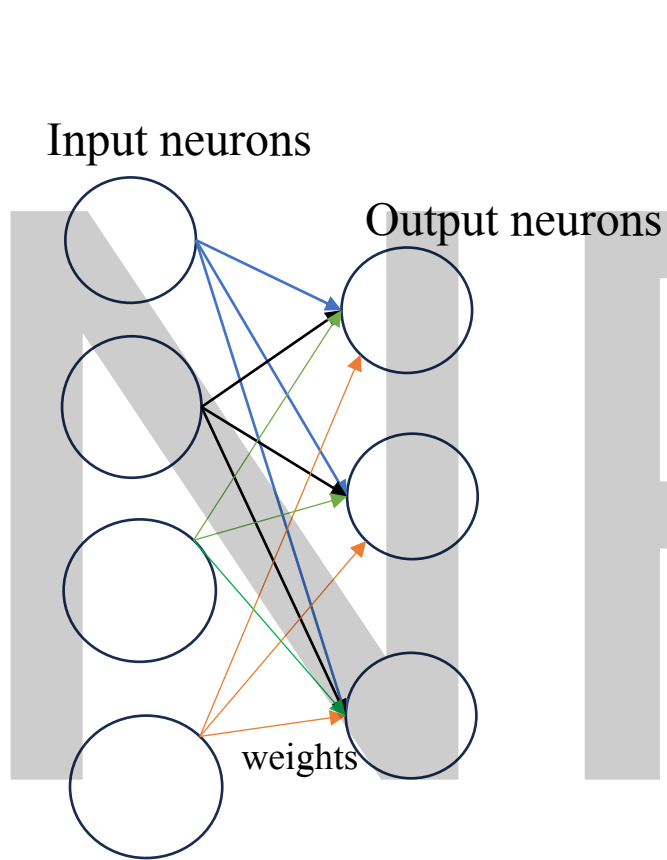
MLP is composed of multiple layers, including an

- input layer
- hidden layers
- output layer

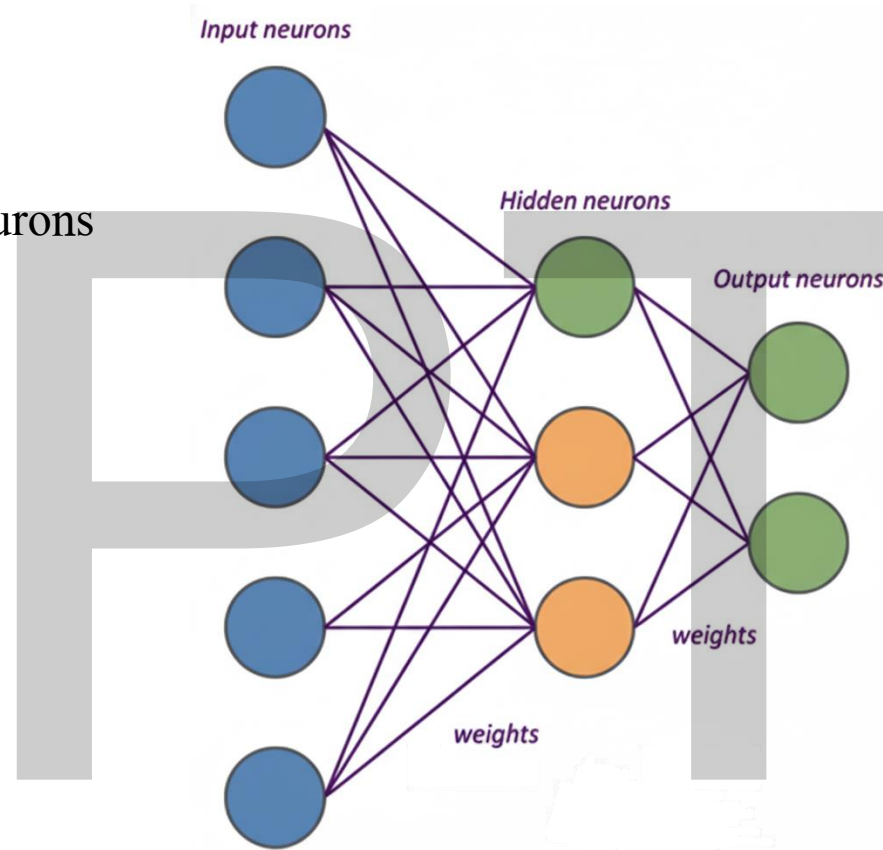
where each layer contains a set of perceptron elements known as neurons.



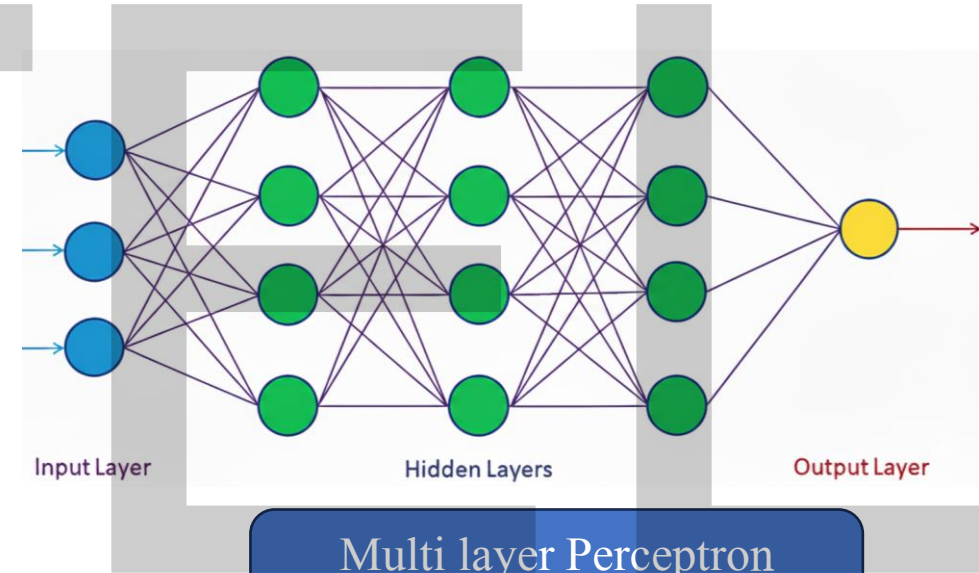
Single layer and Multilayer Perceptron- Comparison



Single layer Perceptron



Multi layer Perceptron
(with only one hidden layer)



Multi layer Perceptron
(with multiple hidden layer)

Why do we need Multilayer Perceptron

Multilayer Perceptron (MLP) is used because it can handle complex datasets that are not linearly separable.

Use cases:

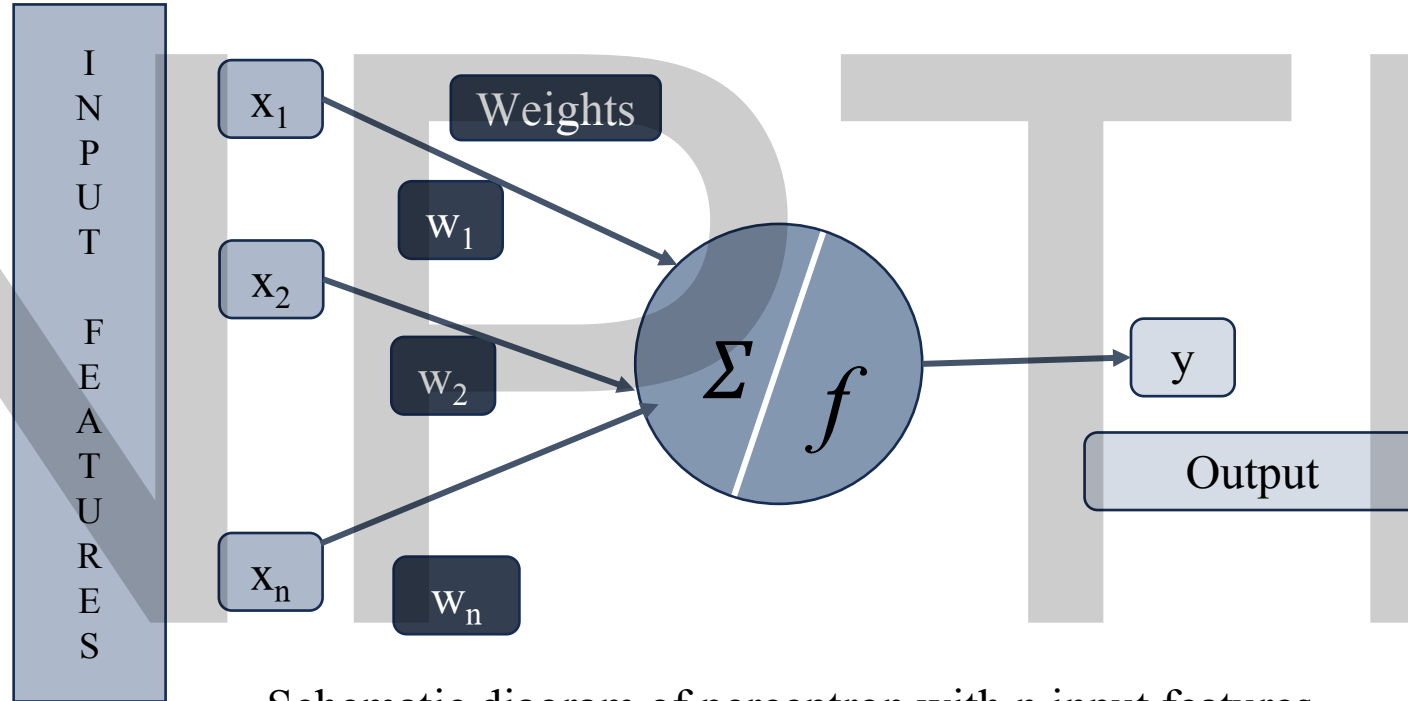
Multilayer perceptrons have been widely used in various domains, such as

- ✓ Image recognition (handwritten digit recognition)
- ✓ Natural language processing (sentiment analysis)
- ✓ Speech recognition (voice to text transcription)

Their ability to learn complex patterns from large volumes of data has made them an essential tool in machine learning.

Working of a Perceptron

A perceptron receives n features as input ($\mathbf{x} = x_1, x_2, \dots, x_n$), and each of these features is associated to a weight.



Schematic diagram of perceptron with n input features

- ✓ Input features must be numeric.
- ✓ Nonnumeric input features have to be converted to numeric ones in order to use a perceptron.

Feed Forward Neural Network

Input Layer

Hidden Layer

Output Layer

Weighted sum of inputs:

$$z_1 = w_{11} * x_1 + w_{21} * x_2 + b_1$$

Activation function:

$$a_1 = g(z_1)$$

Weighted sum of inputs:

$$z_3 = w_{1k} * a_1 + w_{2k} * a_2$$

Activation function:

$$a_3 = g(z_3)$$

Output = a_3

Target = y

Loss (L) = $f(y, a_3)$

Activation function:

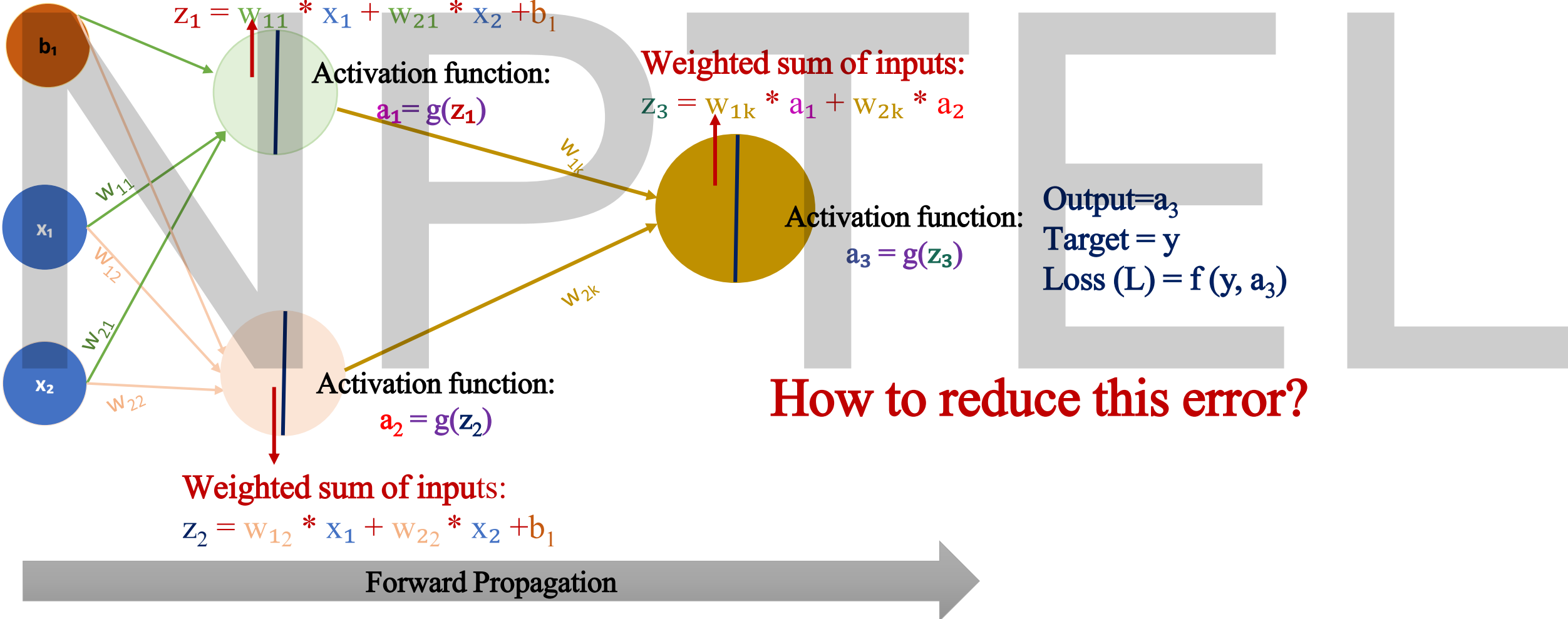
$$a_2 = g(z_2)$$

Weighted sum of inputs:

$$z_2 = w_{12} * x_1 + w_{22} * x_2 + b_1$$

How to reduce this error?

Forward Propagation



Feed Forward Neural Network- Numerical Example

Neuron 1

Suppose inputs $x_1=1$

$$x_2=2$$

Bias $b_1=0.5$

Activation function used: Sigmoid

$$g(x)=\frac{1}{1+e^{-x}}$$

$$\begin{aligned} z_1 &= w_{11}x_1 + w_{21}x_2 + b_1 \\ &= 0.8 * 1.0 + (-0.5) * 2.0 + 0.5 \\ &= 0.8 - 1.0 + 0.5 = 0.3 \\ a_1 &= g(0.3) = \frac{1}{1+e^{-0.3}} = 0.574442516812 \end{aligned}$$

Neuron 2

$$\begin{aligned} z_2 &= w_{12}x_1 + w_{22}x_2 + b_1 \\ &= 0.3 * 1.0 + 1.0 * 2.0 + 0.5 \\ &= 0.3 + 2.0 + 0.5 = 2.8 \\ a_2 &= g(2.8) = \frac{1}{1+e^{-2.8}} = 0.942675824101 \end{aligned}$$

Output Neuron

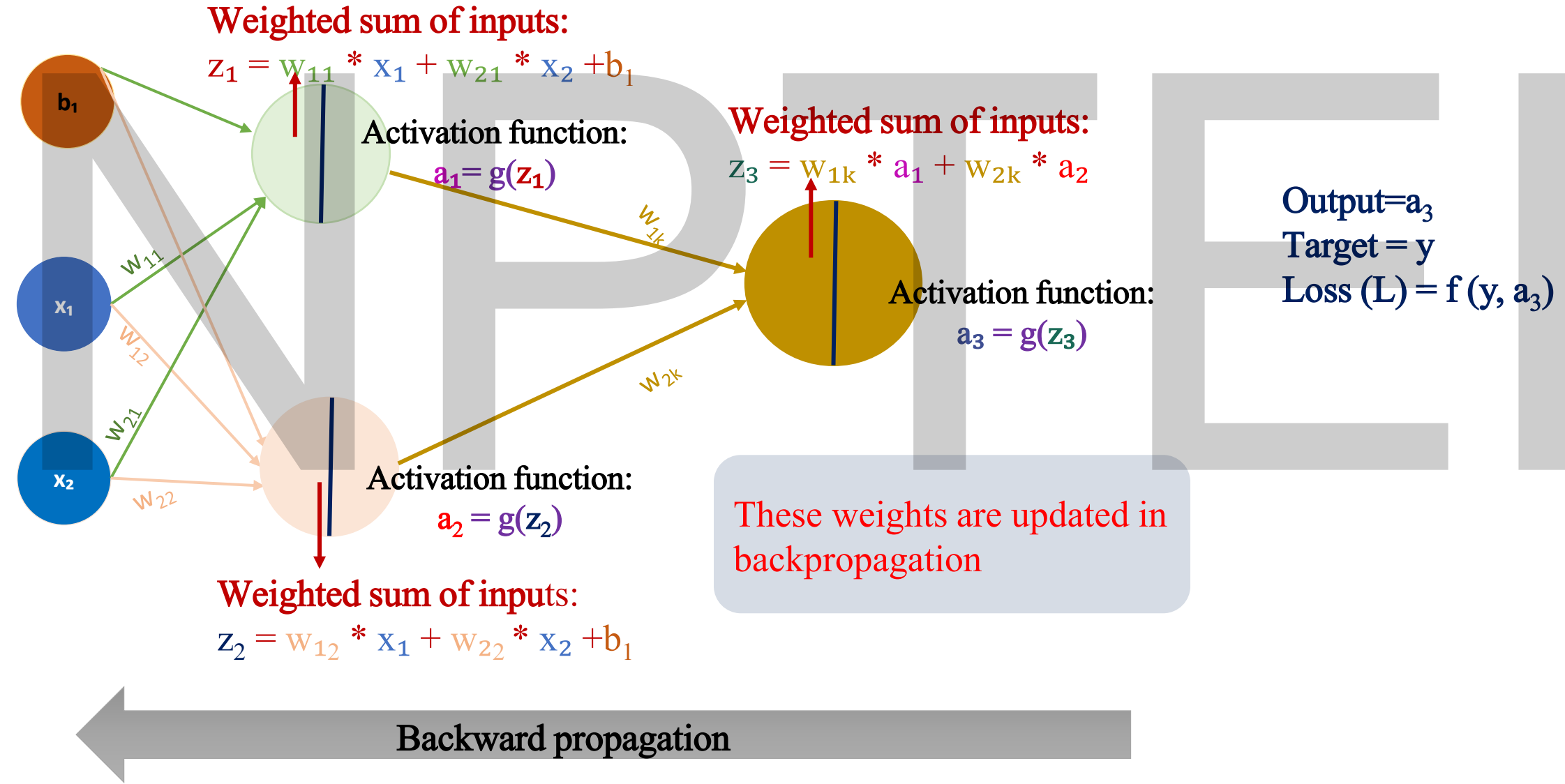
$$\begin{aligned} z_3 &= w_{1k}a_1 + w_{2k}a_2 \\ &= 1.2 * 0.574442516812 + (-0.7) * 0.942675824101 = 0.029457943303 \\ a_3 &= g(0.02945) = \frac{1}{1+e^{-0.02954}} = 0.942675824101 \end{aligned}$$

Feed Forward Neural Network

Input Layer

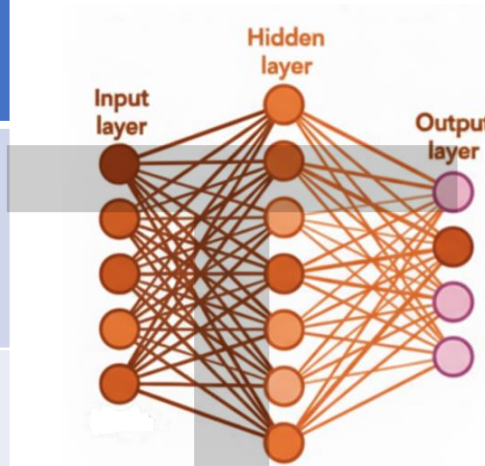
Hidden Layer

Output Layer

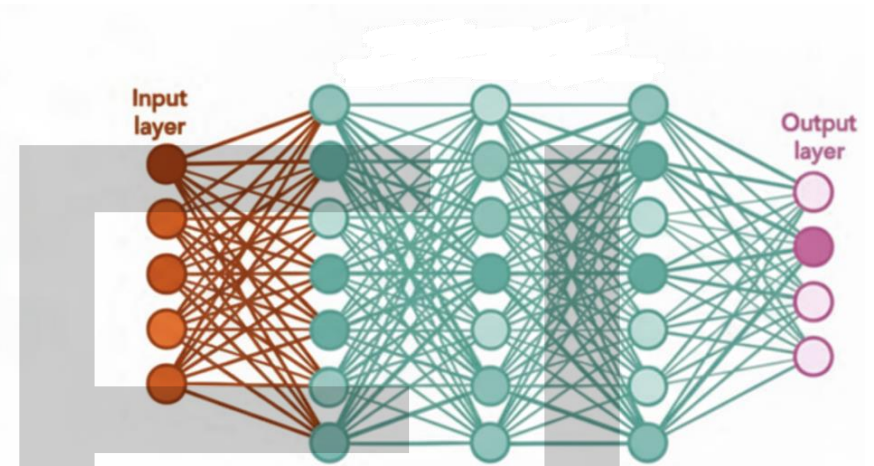


Shallow Neural Networks and Deep Neural Network

Aspect	Shallow Neural Network	Deep Neural Network
Number of Layers	one hidden layer between the input and output layers.	3 or more hidden layers
Handling Complexity	Good for simple or linearly separable problems	Excellent at handling complex, non-linear patterns
Applications	Linear regression, simple binary classification etc	Image and speech recognition, natural language processing, complex predictive analytics.
Feature learning	Limited capability due to a smaller number of layers	Capable of learning complex features



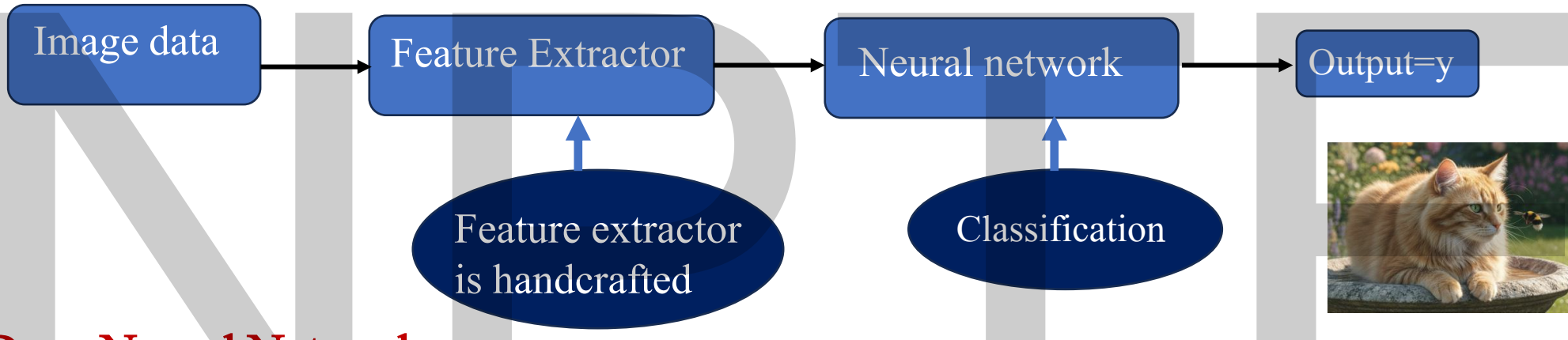
Shallow Neural network



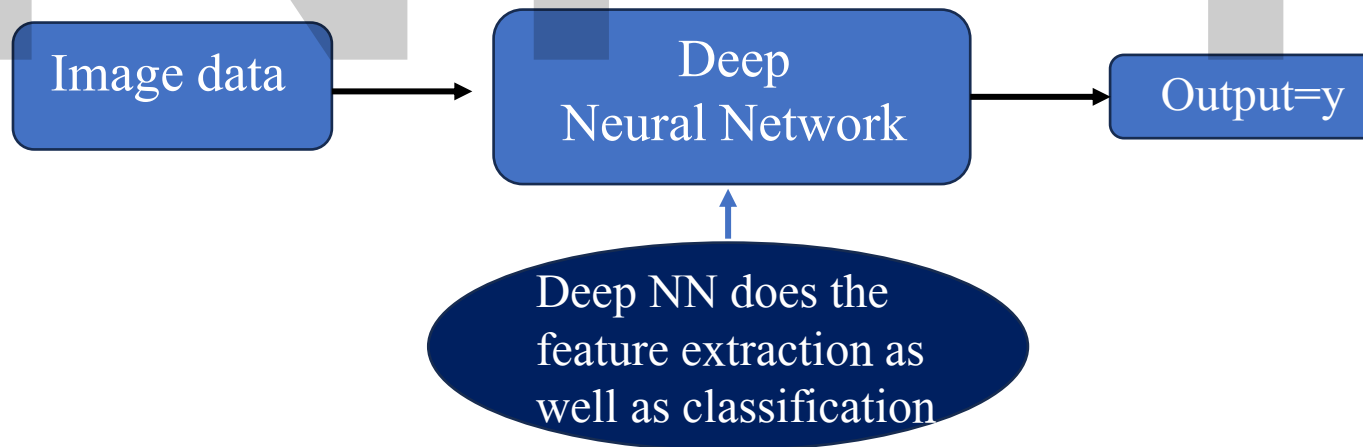
Deep Neural network

Feature extraction using Shallow and Deep Neural Network for a classification problem

Shallow Neural Network



Deep Neural Network



Summary

- ✓ Multilayer perceptron- Difference between Single layer and multilayer
- ✓ Feed Forward Neural Network
- ✓ Numerical on Feed forward neural network
- ✓ Shallow vs Deep Neural Networks

NPTEL

Next Session

Activation function and Loss functions

NPTTEL

Thank you